

1. Technical Details

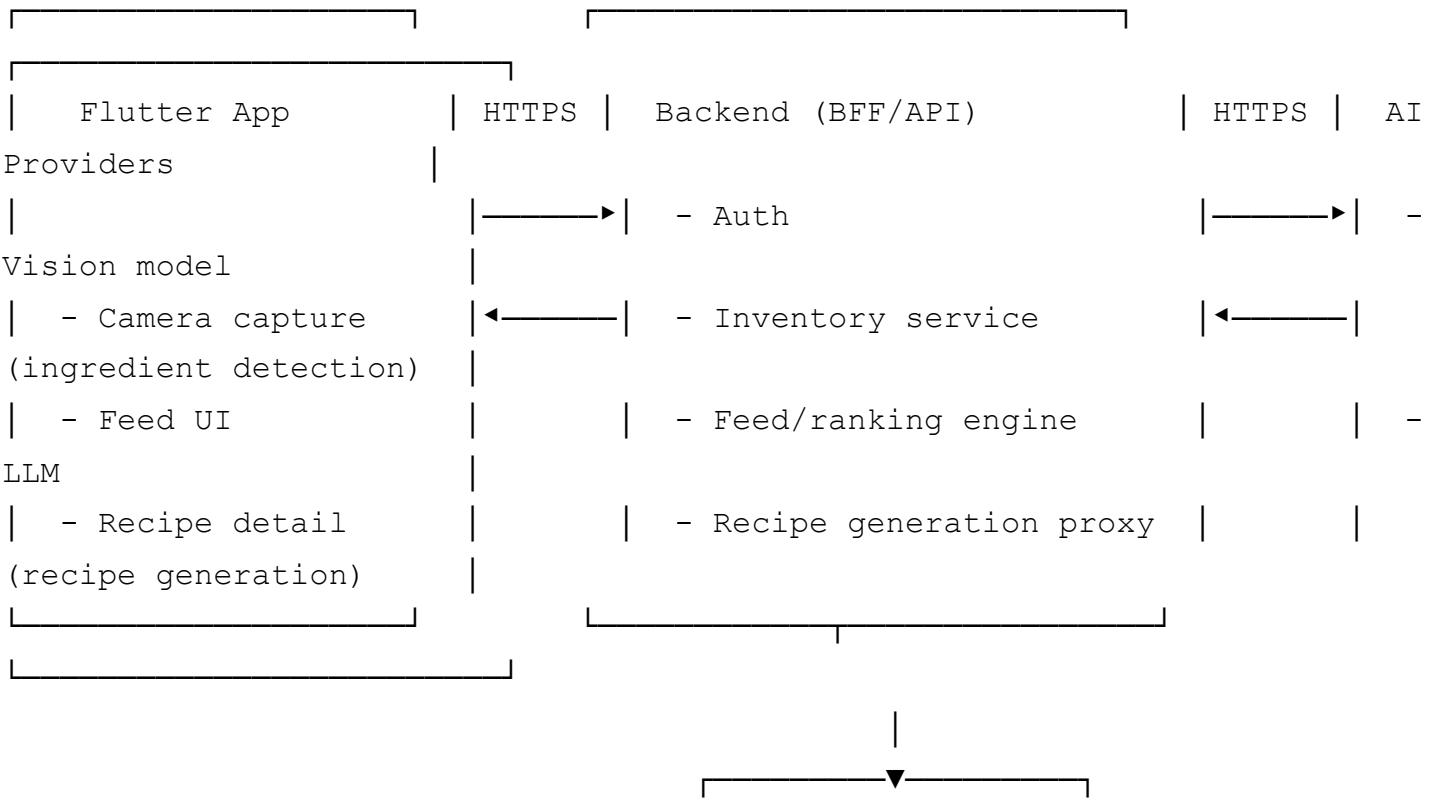
1.1 Recommended build sequencing (not all layers ship at once)

Given the scope (vision AI, LLM generation, expiry prediction, and a UGC/social system are each substantial), the realistic path is phased:

Stage	Scope	Validates
MVP	Manual + photo-scan inventory, LLM-generated recipes, reactive feed	Do people want a feed over a search box? Is LLM recipe quality good enough to replace a database?
v1.5	Receipt scan, expiry prediction	Does auto-inventory reduce the drop-off that kills every competitor?
v2	Community/UGC feed, voting, ranking	Does the app become social/sticky, or stay a solo tool?

The rest of this section covers the **MVP** architecture in depth, with v1.5/v2 noted as extension points.

1.2 System architecture (MVP)



	Database	
	- users	
	- inventory items	
	- generated recipes	
	(cache)	
	- feed state	
└──┘		

Why a backend is non-negotiable here (not optional like a simple CRUD app):

- Vision and LLM API keys must never live in the client
- Recipe generation needs prompt engineering, output validation, and caching server-side (LLM calls are costly – you don't want to regenerate identical recipes on every app open)
- Feed ranking logic (match %, expiry urgency) needs a consistent source of truth, not per-device logic

1.3 Core technical components

a) Ingredient detection (photo scan)

- User photo → sent to backend → backend calls a vision-capable model (e.g. GPT-4V/Gemini-class multimodal model) with a structured-output prompt
- Model returns a list of `{ ingredient, estimated_quantity, confidence }`
- Low-confidence detections are flagged in-app for quick user confirmation (never silently trust a guess – builds trust and improves data quality over time)

b) Recipe generation (LLM)

- Backend sends current inventory + user preferences (diet, allergies, cook time) to an LLM with a **structured JSON output prompt** (title, ingredients used, ingredients needed, steps, estimated time)
- Output is validated against a schema before being shown to the user (reject/retry on malformed output)
- Generated recipes are cached (keyed on inventory hash + preferences) for a short TTL to avoid regenerating identical results and to control cost
- Optionally, blend in a small curated recipe set for cold-start users with empty/near-empty inventories

c) Feed ranking engine

Ranks generated recipe cards by a weighted score, e.g.:

```
score = (ingredient_match_percent * 0.5)
        + (expiry_urgency_score * 0.3)
        + (user_preference_fit * 0.2)
```

Recomputed and re-served every time inventory changes – this is what makes the "feed" feel alive rather than a one-time result set.

d) Expiry prediction (v1.5)

- MVP: static shelf-life lookup table per ingredient category (e.g. leafy greens ~5 days, root vegetables ~3 weeks)
- Later: personalize using the user's own historical "marked as used/wasted" data as a lightweight ML signal

e) Community feed (v2)

- Standard UGC pipeline: submission → moderation (automated content filter + report flow) → ranking (votes, recency-weighted)
- Kept fully separate from the personal feed data model so it can be built independently without touching the core loop

1.4 Suggested stack

Layer	Choice	Why
Frontend	Flutter (Dart)	Cross-platform, camera + push notification support mature
State management	Riverpod	Testable, works well with async feed-refresh patterns
Networking	<code>dio</code> + typed API client	Interceptors for auth, retry, caching
Local cache	<code>drift</code> (SQLite)	Offline view of last-known inventory/feed
Backend	NestJS (Node/TS)	Structured service boundaries (inventory, feed, generation)
Backend DB	PostgreSQL	Relational fit for users, inventory, recipe cache
Vision model	GPT-4V/Gemini-class multimodal API	Ingredient detection from photos
LLM	Claude/GPT-class model via API	Structured recipe generation

Layer	Choice	Why
Push notifications	Firebase Cloud Messaging	Expiry alerts, feed-refresh nudges
Auth	Firebase Auth or Supabase Auth	Fast integration, social login support
Analytics	PostHog or Firebase Analytics	Track scan→feed→cook conversion funnel

1.5 Core API contracts (MVP)

POST `/inventory/scan`

Body: multipart image

```
{
  "detected": [
    { "id": "tmp_1", "name": "Tomato", "quantity_estimate": "3 pcs", "correlation": 0.95 },
    { "id": "tmp_2", "name": "Spinach", "quantity_estimate": "1 bag", "correlation": 0.9 }
  ]
}
```

POST `/inventory` (confirm/manual add)

```
{ "items": [{ "name": "Tomato", "quantity": "3", "unit": "pcs" }] }
```

GET `/feed`

Returns the ranked, ready-to-render feed for the current inventory state.

```
{
  "feed": [
    {
      "recipeId": "gen_88f1",
      "title": "Spinach & Tomato Shakshuka",
      "matchPercent": 90,
      "expiryFlag": "spinach expires in 2 days",
      "missingIngredients": ["feta cheese"],
      "cookTimeMinutes": 25
    }
  ]
}
```

GET /recipes/:id

```
{
  "id": "gen_88f1",
  "title": "Spinach & Tomato Shakshuka",
  "servings": 2,
  "ingredients": [
    { "name": "Tomato", "quantity": "3", "unit": "pcs", "have": true },
    { "name": "Feta cheese", "quantity": "50", "unit": "g", "have": false }
  ],
  "steps": [
    { "order": 1, "text": "Dice the tomatoes and wilt the spinach in a pan." },
    { "order": 2, "text": "Crack eggs into the pan and simmer until set." }
  ]
}
```

1.6 Key technical risks to plan for early

- **Vision detection accuracy** – cluttered fridges, poor lighting, and packaging-only-visible items (can't see what's inside a container) will produce false positives/negatives; the confirm-before-commit UX step is essential, not optional
 - **LLM recipe quality/consistency** – needs strong prompt constraints + output validation to avoid nonsensical or unsafe combinations (e.g. undercooked meat instructions); consider a lightweight safety/sanity-check pass on generated steps
 - **Cost control** – vision + LLM calls are the most expensive part of the stack; caching, rate-limiting scans, and reusing generated recipes across similar inventories will matter a lot at scale
 - **Cold start** – a brand-new user with an empty inventory needs a fallback experience (curated starter recipes, prompt to do first scan) so the feed isn't empty on day one
-